Université Savoie Mont Blanc, UFR SceM
INFO702, M1 INFO

November 9, 2026

TD2 : programmation générique

Fonctions et classes génériques, itérateurs, foncteurs, spécialisation

Les exemples sont en C++11. Les noms de fonctions réimplémentées sont placés dans l'espace de noms `td` afin de distinguer leurs appels de ceux de la bibliothèque standard. Un intervalle `[first,last)` inclut son début et exclut sa fin ; on ne déréférence jamais `last`. Quand on demande les exigences sur les types, on attend d'abord les opérations et propriétés nécessaires, sans exiger de syntaxe `requires`. Les exercices 1 à 8 constituent la progression principale ; les trois approfondissements peuvent être traités séparément.

Question 1. Fonction générique : référencer le maximum de trois valeurs

On veut que le code suivant fasse désigner la variable `y` par `t` :

```
double x = 5.7, y = 10.2, z = 9.4;
double& t = ref_max(x, y, z);
```

1. Écrire le patron de fonction `ref_max`. Les trois arguments sont des variables modifiables de même type. On utilise seulement `operator<` pour les comparer ; en cas d'égalité, on retient le premier maximum dans l'ordre des arguments.
2. Pourquoi retourner une référence ? Quel effet aurait `t = 0` ? Pourquoi `ref_max(1,2,3)` ne doit-il pas être accepté par cette version ?

Question 2. Fonction générique : argument minimum d'un intervalle

On part de la déclaration `int* argmin(int* deb, int* fin)`. L'intervalle considéré est toujours semi-ouvert : `[deb,fin)`.

1. Écrire une version générique qui renvoie un itérateur vers le premier minimum, ou `fin` si l'intervalle est vide. Quelles opérations sont nécessaires sur les itérateurs et les valeurs ? Pourquoi le parcours doit-il être multipasse ?
2. Définir la comparaison manquante pour trouver l'hôtel le moins cher :

```
struct Hotel { std::string name; double prix_par_nuit; };
Hotel hotels[] = {{"Lac", 95.0}, {"Alpes", 80.0}, {"Parc", 110.0}};
auto it = argmin(std::begin(hotels), std::end(hotels));
if (it != std::end(hotels))
  std::cout << it->name << " : " << it->prix_par_nuit << '\n';
```

3. Tester aussi avec une `std::list<int>` et une collection vide. Que signifie `const auto it` ? Cela rend-il l'élément désigné constant ?

Question 3. Échange et inversion d'un intervalle

1. Écrire `echange(T&,T&)` avec une variable temporaire et des copies. Quelles opérations doit fournir `T` ?
2. Écrire `inversion(first,last)`, qui inverse les éléments de `[first,last)`. Traiter les intervalles vides et de taille un. Quelle catégorie d'itérateurs est nécessaire ? Tester un tableau et une liste.
3. En complément, adapter l'échange pour un type déplaçable mais non copiable.

Question 4. Comparateur d'un ensemble ordonné

1. Qu'affiche le code suivant ?

```
std::set<int> s{3, 12, 7, 1, 15};
for (auto v : s) std::cout << ' ' << v;
```

2. Écrire le foncteur `Greater` pour obtenir l'ordre décroissant. On utilisera ensuite `std::set<int,Greater>`. La comparaison doit être appelable sur un objet constant. Pourquoi `i >= j` ne convient-il pas ?

3. Que se passe-t-il si on insère deux fois la même valeur ?

Question 5. Foncteurs et expressions lambda

On souhaite conserver les valeurs d'un intervalle fermé $[a, b]$, avec $a \leq b$.

1. Écrire `IsInInterval`, qui renvoie vrai pour une valeur dans cet intervalle, puis compléter l'appel suivant. Attention : `remove_if` retire les valeurs pour lesquelles son prédicat vaut vrai.

```
double values[] = {-3.5, 15.4, 6.4, -17.2, 2.1, 25.2, 30.2};
IsInInterval filter(5.0, 19.0);
auto last = std::remove_if(std::begin(values), std::end(values), ...);
for (auto it = std::begin(values); it != last; ++it)
  std::cout << ' ' << *it; // affiche 15.4 6.4
```

   La taille physique du tableau a-t-elle changé ?

2. Remplacer le foncteur par une lambda capturant les bornes par valeur.

3. Sur un nouveau tableau contenant les sept valeurs initiales, utiliser `std::for_each` et des lambdas pour calculer le nombre de valeurs strictement positives, le maximum et la moyenne. Expliquer les captures. Que faudrait-il préciser si l'intervalle était vide ?

Question 6. Partition d'une collection

Écrire `partition(first,last,pred)`, qui place au début les éléments satisfaisant le prédicat, puis les autres. Le résultat désigne le premier élément du second groupe, éventuellement `last`. L'ordre interne des groupes n'est pas imposé.

```
int t[] = {17, 4, 8, 5, 13, 15, 1, 20, 13, 12, 9};
auto p = partition( std::begin(t), std::end(t),
                    [](int v) { return v <= 10; } );
// Cinq valeurs <= 10 dans [begin(t),p), les autres dans [p,end(t)).
```

1. Proposer un algorithme linéaire utilisant seulement des itérateurs forward. On peut employer `std::iter_swap`, mais pas `std::partition`. Préciser l'invariant et les exigences sur les types.

2. Tester l'intervalle vide et les cas où tous les éléments, ou aucun, satisfont le prédicat. La partition est-elle stable ?

3. Quicksort utilise une partition. L'accès aléatoire est-il nécessaire pour cette partition ? Quel intérêt a-t-il pour certaines implémentations du tri ?

Question 7. Classe générique : Tuple et spécialisation partielle

On souhaite représenter N valeurs de même type `T`, avec $1 \leq N \leq 32$, où `N` et `T` sont fixés à l'instanciation.

1. Écrire `Tuple<N,T>`, initialisé par valeur, avec `size()` et deux surcharges de `operator[]` pour la lecture et l'écriture. L'indice doit appartenir à `[0,N)`.

2. Spécialiser partiellement `Tuple<N,bool>` pour stocker les bits dans un `std::uint32_t` de `<cstdint>`, supposé disponible. Fournir d'abord `bool operator[](std::size_t) const` et `set(std::size_t,bool)`. Pourquoi ne peut-on pas renvoyer un `bool&` ?

3. En complément, proposer un objet proxy pour autoriser `t[i]=true`.

Question 8. Un premier itérateur : intervalle d'entiers

On souhaite parcourir les entiers comme avec `range` en Python :

```
Range r(3,8,2);
for (int v : r) std::cout << ' ' << v; // 3 5 7
```

1. Proposer `Range` et un itérateur compatible avec la boucle ci-dessus. L'intervalle est semi-ouvert : un pas positif progresse tant que la valeur est strictement inférieure à la borne finale ; un pas négatif progresse tant qu'elle lui est strictement supérieure. Un pas nul est rejeté. Une direction incompatible donne un intervalle vide. On suppose les paramètres et toutes les valeurs produites représentables par `int` ; éviter également le débordement pendant le calcul de l'incrément suivant.

2. Ajouter un patron de classe `FilteredRange<Pred>` qui ne produit que les entiers satisfaisant un prédicat. On peut se limiter à un curseur permettant la boucle `for`, sans revendiquer toutes les exigences STL. Le prédicat est copiable, appelable sur un objet constant, et ne modifie pas son argument. Tester un prédicat qui ne retient aucune valeur.

Question 9. Approfondissement : liste doublement chaînée

Implémenter une version minimale `Liste<T>` avec une sentinelle circulaire ne contenant pas de valeur de type `T`.

1. Fournir construction vide, destruction, `empty()`, `begin()`, `end()`, `insert(pos,value)` et `erase(pos)`. L'insertion se fait avant `pos` et renvoie la position créée ; la suppression renvoie la position suivante. Un `push_back` peut être déduit de ces opérations.

2. Écrire un itérateur bidirectionnel mutable, avec types associés, `*`, `->`, `++`, `--` et comparaisons. La sentinelle représente `end()` : elle n'est pas déréférençable. L'opération `--end()` est valide sur une liste non vide.

3. Interdire explicitement copie et déplacement dans cette version. Justifier la complexité constante des modifications lorsque la position est connue et préciser l'invalidation des itérateurs. On ne demande pas ici les parcours d'une liste constante ni l'interface complète de `std::list`.

Question 10. Approfondissement : arbre binaire et arbre de recherche

1. Écrire `Arbre<T>`, vide ou à une valeur, avec `size()`, un curseur sur la racine, l'accès aux fils et le remplacement d'un fils par un noeud. Le remplacement détruit le sous-arbre précédent. Définir une propriété exclusive des noeuds et interdire copie et déplacement.

2. Parcourir l'arbre en ordre *infixe* (gauche, racine, droite) avec un itérateur forward de lecture seule, équipé des types associés. On peut conserver dans chaque noeud un pointeur non propriétaire vers son père.

3. Écrire `ABR<T,Compare>` par composition avec `Arbre<T>`. Les valeurs ne doivent pas être modifiables par l'utilisateur à travers les itérateurs de l'ABR. Le comparateur définit un ordre strict faible ; les doublons sont conservés et insérés à droite en cas d'équivalence.

4. Trier un intervalle modifiable en insérant ses valeurs dans un ABR vide, puis en recopiant son parcours infixe. Donner les complexités en fonction de la hauteur de l'arbre, puis dans le pire cas. On ne demande pas d'équilibrage. Préciser l'effet du remplacement d'un sous-arbre sur les itérateurs.

Question 11. Approfondissement : pointeur à propriété partagée

Écrire une version pédagogique `Shared<T>` qui compte les propriétaires d'un objet alloué par `new T(...)`. Le dernier propriétaire le détruit.

```
struct Object {
  std::string name;
  explicit Object(std::string n) : name(n) {}
};
Shared<Object> p1(new Object("partage"));
Shared<Object> p2=p1;
p2->name="modifie";
std::cout << p1->name; // modifie
```

1. Prévoir un bloc de contrôle partagé avec le pointeur et son compteur. Fournir construction vide, acquisition explicite d'un pointeur, construction par copie, affectation par copie, destruction, `get()`, `use_count()`, `operator*`, `operator->` et conversion explicite en booléen.

2. Traiter l'auto-affectation, l'affectation entre propriétaires du même objet et l'échec de construction du bloc de contrôle. Ajouter le déplacement.

3. Tester le partage de modifications et la destruction unique de l'objet. Pourquoi ne faut-il pas créer deux blocs indépendants pour le même pointeur ? Pourquoi un cycle de propriétaires peut-il empêcher la libération ?

On exclut les tableaux, les conversions entre types, les destructeurs personnalisés et les accès concurrents. Les destructeurs ne lancent pas d'exception. En pratique on utilise `std::shared_ptr` et `std::make_shared`, et non cette réimplémentation.